AE498-Planetary Defense - HW1

Sanyu Kumaran

February 2026

1 Algorithm Overview

The Minimum Orbit Intersection Distance (MOID) was computed using a numerical optimization approach. The position vectors for each orbit were first constructed from their Keplerian orbital elements using the standard inverse transformation from orbital elements to Cartesian state vectors (given in the slides)

For a non-parabolic orbit, the semi-latus rectum is computed as

$$p = a(1 - e^2), \tag{1}$$

and the radial distance is given by

$$r = \frac{p}{1 + e\cos f}, \tag{2}$$

where a is the semi-major axis, e is the eccentricity, and f is the true anomaly.

The position and velocity vectors in the perifocal (PQW) frame are

$$\mathbf{r}_{PQW} = \frac{p}{1 + e\cos f}\begin{bmatrix}\cos f\\ \sin f\\ 0\end{bmatrix}, \tag{3}$$

$$\mathbf{v}_{PQW} = \sqrt{\frac{\mu}{p}}\begin{bmatrix}-\sin f\\ e + \cos f\\ 0\end{bmatrix}, \tag{4}$$

where μ is the gravitational parameter of the central body.

The transformation to the inertial frame is performed using the rotation matrix

$$\mathbf{Q} = \mathbf{R}_3(\Omega)\mathbf{R}_1(i)\mathbf{R}_3(\omega), \tag{5}$$

such that

$$\mathbf{r} = \mathbf{Q}\mathbf{r}_{PQW}, \qquad \mathbf{v} = \mathbf{Q}\mathbf{v}_{PQW}, \tag{6}$$

where Ω is the right ascension of the ascending node, i is the inclination, and ω is the argument of periapsis.

The rotation matrices are Direct Cosine Matrices (DCS) and are given below:

Rotation about the z-axis by the right ascension of the ascending node Ω:

$$\mathbf{R}_3(\Omega) = \begin{bmatrix} \cos\Omega & \sin\Omega & 0 \\ -\sin\Omega & \cos\Omega & 0 \\ 0 & 0 & 1 \end{bmatrix}. \tag{7}$$

Rotation about the x-axis by the inclination i:

$$\mathbf{R}_1(i) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos i & \sin i \\ 0 & -\sin i & \cos i \end{bmatrix}. \tag{8}$$

Rotation about the z-axis by the argument of periapsis ω:

$$\mathbf{R}_3(\omega) = \begin{bmatrix} \cos\omega & \sin\omega & 0 \\ -\sin\omega & \cos\omega & 0 \\ 0 & 0 & 1 \end{bmatrix}. \tag{9}$$

The full direction cosine matrix is therefore

$$\mathbf{Q} = \mathbf{R}_3(\Omega)\mathbf{R}_1(i)\mathbf{R}_3(\omega). \tag{10}$$

The MOID was obtained by minimizing the squared Euclidean distance between two orbital position vectors:

$$J(f_1, f_2) = \|\mathbf{r}_1(f_1) - \mathbf{r}_2(f_2)\|^2, \tag{11}$$

where f_1 and f_2 are the true anomalies of the two orbits and the variables that are being varied. f_1 and f_2 are reported in degrees

Because this objective function is non-convex, multiple stationary points exist corresponding to local minima and saddle points. To mitigate convergence to non-global solutions, a multi-start strategy was implemented. A coarse grid of initial conditions spanning the domain was generated, and the gradient-based optimization function from SciPy was executed from each grid point. A total of 225 initial guesses were used. The global minimum was selected as the smallest value among all converged solutions.

Due to the smooth geometric structure of the orbital distance function, this sampling density was sufficient to reliably identify the global minimum. The computed MOID values were validated by comparison with those reported by NASA Jet Propulsion Laboratory

The link to the github can be found below:

https://github.com/pksanyu787/AE498—Planetary-Defense—Spring-2026

In [1]:
```
import numpy as np
from scipy.optimize import minimize
```

Setup Code

In [2]:
```
# ========================
# EARTH
# ========================
a_Earth = 1.4765067e8              # km
e_Earth = 9.1669995e-03
i_Earth = np.deg2rad(4.2422693e-03)
omega_Earth = np.deg2rad(6.64375167e+01)
Omega_Earth = np.deg2rad(1.4760836e+01)

# ========================
# APOPHIS
# ========================
a_Apophis = 1.3793939e8            # km
e_Apophis = 1.9097084e-01
i_Apophis = np.deg2rad(3.3356539e+00)
omega_Apophis = np.deg2rad(1.2919949e+02)
Omega_Apophis = np.deg2rad(2.0381969e+02)

# ========================
# 2024 YR4
# ========================
a_YR4 = 3.7680703e8                # km
e_YR4 = 6.6164147e-01
i_YR4 = np.deg2rad(3.4001497e+00)
omega_YR4 = np.deg2rad(1.3429905e+02)
Omega_YR4 = np.deg2rad(2.7147904e+02)

# ========================
# 3I/ATLAS
# ========================
a_ATLAS = -3.9552667e7             # km (hyperbolic, negative a)
e_ATLAS = 6.1469268e+00
i_ATLAS = np.deg2rad(1.7512507e+02)
omega_ATLAS = np.deg2rad(1.2817255e+02)
Omega_ATLAS = np.deg2rad(3.2228906e+02)
```

In [3]:
```
def kmToAU(km):
    return km / 1.496e+8

def R3_omega(omega):
    """
    Rotation about Z-axis by omega
    """
    return np.array([
        [ np.cos(omega),  np.sin(omega), 0.0],
        [-np.sin(omega),  np.cos(omega), 0.0],
        [ 0.0,            0.0,           1.0]
    ])
```

```
def R1_inclination(inclination):
    """
    Rotation about X-axis by inclination
    """
    return np.array([
        [1.0, 0.0,                 0.0],
        [0.0, np.cos(inclination),  np.sin(inclination)],
        [0.0, -np.sin(inclination), np.cos(inclination)]
    ])


def R3_periapsis(omega_small):
    """
    Rotation about Z-axis by argument of periapsis (little omega)
    """
    return np.array([
        [ np.cos(omega_small),  np.sin(omega_small), 0.0],
        [-np.sin(omega_small),  np.cos(omega_small), 0.0],
        [ 0.0,                  0.0,                 1.0]
    ])
```

In [4]:

```
def r_vec(a, e, v):
    r = a * (1 - e**2) / (1 + e * np.cos(v))
    r_vec = np.array([r * np.cos(v), r * np.sin(v), 0])
    return r_vec


def r_inertial(omega, inclination, omega_small, a, e, v):
    R_omega = R3_omega(omega)
    R_inclination = R1_inclination(inclination)
    R_periapsis = R3_periapsis(omega_small)

    vector = r_vec(a, e, v)
    return R_omega @ R_inclination @ R_periapsis @ vector
```

In [25]:

```
def r_earth(f):
    return r_inertial(Omega_Earth, i_Earth, omega_Earth,
                      a_Earth, e_Earth, f)

def r_apophis(f):
    return r_inertial(Omega_Apophis, i_Apophis, omega_Apophis,
                      a_Apophis, e_Apophis, f)

def r_Asteroid2024(f):
    return r_inertial(Omega_YR4, i_YR4, omega_YR4, a_YR4, e_YR4, f)

def r_3IAtlas(f):
    return r_inertial(Omega_ATLAS, i_ATLAS, omega_ATLAS, a_ATLAS, e_ATLAS, f)


def distance_squared(vars):
    f1, f2 = vars
```

```
    r1 = r_earth(f1)
    r2 = r_apophis(f2)

    diff = r1 - r2
    return np.dot(diff, diff)
```

Earth and Asteroid 99942 Apophis

In [19]:
```
bounds = [(0, 2*np.pi), (0, 2*np.pi)]

best_val = np.inf
best_sol = None

for f1_guess in np.linspace(0, 2*np.pi, 15):
    for f2_guess in np.linspace(0, 2*np.pi, 15):

        result = minimize(distance_squared,
                          x0=[f1_guess, f2_guess],
                          bounds=bounds)

        if result.fun < best_val:
            best_val = result.fun
            best_sol = result.x

f1_opt, f2_opt = best_sol
moid_km = np.sqrt(best_val)

print("MOID in Km for Earth and 99942 Apophis", moid_km)
print("MOID in AU for Earth and 99942 Apophis", kmToAU(moid_km))
print("f1 for Earth and 99942 Apophis", np.rad2deg(f1_opt))
print("f2 for Earth and 99942 Apophis", np.rad2deg(f2_opt))
```

```
MOID in Km for Earth and 99942 Apophis 847896.6685519756
MOID in AU for Earth and 99942 Apophis 0.005667758479625505
f1 for Earth and 99942 Apophis 232.03256883297362
f2 for Earth and 99942 Apophis 123.84505283924122
```

Earth and Asteroid 2024 YR4

In [20]:
```
def distance_squared_Earth_and_asteroid(vars):
    f1, f2 = vars

    r1 = r_earth(f1)
    r2 = r_Asteroid2024(f2)

    diff = r1 - r2
    return np.dot(diff, diff)
```

In [21]:
```
bounds = [(0, 2*np.pi), (0, 2*np.pi)]

best_val = np.inf
best_sol = None

for f1_guess in np.linspace(0, 2*np.pi, 15):
    for f2_guess in np.linspace(0, 2*np.pi, 15):
```

```
        result = minimize(distance_squared_Earth_and_asteroid,
                          x0=[f1_guess, f2_guess],
                          bounds=bounds)

        if result.fun < best_val:
            best_val = result.fun
            best_sol = result.x

f1_opt, f2_opt = best_sol
moid_km = np.sqrt(best_val)

print("MOID in Km for Earth and Asteroid 2024 YR4", moid_km)
print("MOID in AU for Earth and Asteroid 2024 YR4", kmToAU(moid_km))
print("f1 for Earth and Asteroid 2024 YR4", np.rad2deg(f1_opt))
print("f2 for Earth and Asteroid 2024 YR4" , np.rad2deg(f2_opt))
```

```
MOID in Km for Earth and Asteroid 2024 YR4 264452.6442741219
MOID in AU for Earth and Asteroid 2024 YR4 0.0017677315793724726
f1 for Earth and Asteroid 2024 YR4 348.0730039964497
f2 for Earth and Asteroid 2024 YR4 312.6498043076265
```

Asteroid 99942 Apophis and asteroid 2024 YR4

In [22]:
```
def distance_squared_Apophis_and_Y24(vars):
    f1, f2 = vars

    r1 = r_apophis(f1)
    r2 = r_Asteroid2024(f2)

    diff = r1 - r2
    return np.dot(diff, diff)
```

In [24]:
```
bounds = [(0, 2*np.pi), (0, 2*np.pi)]

best_val = np.inf
best_sol = None

for f1_guess in np.linspace(0, 2*np.pi, 15):
    for f2_guess in np.linspace(0, 2*np.pi, 15):

        result = minimize(distance_squared_Apophis_and_Y24,
                          x0=[f1_guess, f2_guess],
                          bounds=bounds)

        if result.fun < best_val:
            best_val = result.fun
            best_sol = result.x

f1_opt, f2_opt = best_sol
moid_km = np.sqrt(best_val)

print("MOID in Km for 99942 Apophis and Asteroid 2024 YR4", moid_km)
print("MOID in AU for 99942 Apophis and Asteroid 2024 YR4", kmToAU(moid_km))
```

```
print("f1 for 99942 Apophis and Asteroid 2024 YR4", np.rad2deg(f1_opt))
print("f2 for 99942 Apophis and Asteroid 2024 YR4" , np.rad2deg(f2_opt))
```

```
MOID in Km for 99942 Apophis and Asteroid 2024 YR4 7530657.644115732
MOID in AU for 99942 Apophis and Asteroid 2024 YR4 0.05033862061574687
f1 for 99942 Apophis and Asteroid 2024 YR4 235.7029335531582
f2 for 99942 Apophis and Asteroid 2024 YR4 308.49397702628215
```

The Earth and 3I/ATLAS

In [26]:
```
def distance_squared_Earth_and_Atlas(vars):
    f1, f2 = vars

    r1 = r_earth(f1)
    r2 = r_3IAtlas(f2)

    diff = r1 - r2
    return np.dot(diff, diff)
```

In [27]:
```
bounds = [(0, 2*np.pi), (0, 2*np.pi)]

best_val = np.inf
best_sol = None

for f1_guess in np.linspace(0, 2*np.pi, 15):
    for f2_guess in np.linspace(0, 2*np.pi, 15):

        result = minimize(distance_squared_Earth_and_Atlas,
                          x0=[f1_guess, f2_guess],
                          bounds=bounds)

        if result.fun < best_val:
            best_val = result.fun
            best_sol = result.x

f1_opt, f2_opt = best_sol
moid_km = np.sqrt(best_val)

print("MOID in Km for Earth and 3I/ATLAS", moid_km)
print("MOID in AU for Earth and 3I/ATLAS", kmToAU(moid_km))
print("f1 for Earth and 3I/ATLAS", np.rad2deg(f1_opt))
print("f2 for Earth and 3I/ATLAS" , np.rad2deg(f2_opt))
```

```
MOID in Km for Earth and 3I/ATLAS 56610142.75120673
MOID in AU for Earth and 3I/ATLAS 0.3784100451283873
f1 for Earth and 3I/ATLAS 247.27041885350278
f2 for Earth and 3I/ATLAS 359.78127470562924
```